

Dynamic Programming – Basic Problems

Dice Throw Problem, Binomial Coefficient, Assembly Line Scheduling

GUIDE: Dr. SRIRAMYA

NAME: JAASIEL J.S

Dice Throw Problem

1. Write a program to determine the number of ways a player can unlock a reward in an online game. The game uses 3 six-faced dice, and the reward is granted only when the total score obtained is exactly 8. Display the total number of possible combinations that produce the target score.

Input:

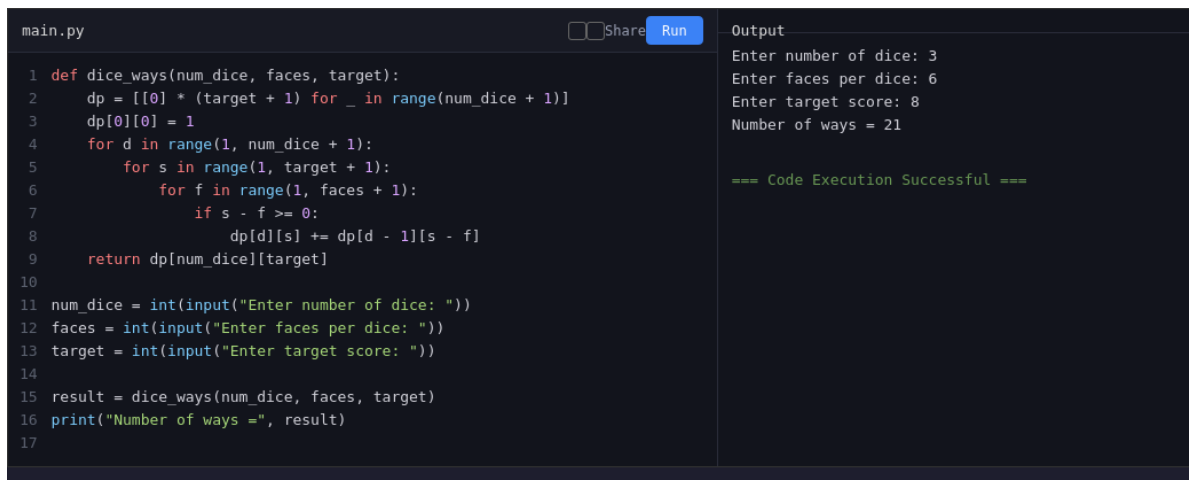
Number of Dice = 3

Faces per Dice = 6

Target Score = 8

Output:

Number of ways = 21



```
main.py
1 def dice_ways(num_dice, faces, target):
2     dp = [[0] * (target + 1) for _ in range(num_dice + 1)]
3     dp[0][0] = 1
4     for d in range(1, num_dice + 1):
5         for s in range(1, target + 1):
6             for f in range(1, faces + 1):
7                 if s - f >= 0:
8                     dp[d][s] += dp[d - 1][s - f]
9     return dp[num_dice][target]
10
11 num_dice = int(input("Enter number of dice: "))
12 faces = int(input("Enter faces per dice: "))
13 target = int(input("Enter target score: "))
14
15 result = dice_ways(num_dice, faces, target)
16 print("Number of ways =", result)
17
```

Output

```
Enter number of dice: 3
Enter faces per dice: 6
Enter target score: 8
Number of ways = 21

=== Code Execution Successful ===
```

Question 2: Board Game Tournament

Write a program to calculate the number of possible outcomes in a board game tournament where a player rolls 2 dice, each having 6 faces. To qualify for the next round, the player must obtain a total score of 7.

Input:

Number of Dice = 2

Faces per Dice = 6

Target Score = 7

Output:

Number of ways = 6

```
main.py Share Run
1 def dice_ways(num_dice, faces, target):
2     dp = [[0] * (target + 1) for _ in range(num_dice + 1)]
3     dp[0][0] = 1
4     for d in range(1, num_dice + 1):
5         for s in range(1, target + 1):
6             for f in range(1, faces + 1):
7                 if s - f >= 0:
8                     dp[d][s] += dp[d - 1][s - f]
9     return dp[num_dice][target]
10
11 num_dice = int(input("Enter number of dice: "))
12 faces = int(input("Enter faces per dice: "))
13 target = int(input("Enter target score: "))
14
15 result = dice_ways(num_dice, faces, target)
16 print("Number of ways =", result)
17
```

Output

```
Enter number of dice: 2
Enter faces per dice: 6
Enter target score: 7
Number of ways = 6

=== Code Execution Successful ===
```

Question 3: Warehouse Robot Navigation

A warehouse robot moves forward based on the values obtained from 4 dice throws. Each dice has 4 faces. The robot reaches its destination only if the sum of all throws equals 10. Write a program to determine the number of possible movement sequences.

Input:

Number of Dice = 4

Faces per Dice = 4

Target Distance = 10

Output:

Number of ways = 44

```
main.py Share Run
1 def dice_ways(num_dice, faces, target):
2     dp = [[0] * (target + 1) for _ in range(num_dice + 1)]
3     dp[0][0] = 1
4     for d in range(1, num_dice + 1):
5         for s in range(1, target + 1):
6             for f in range(1, faces + 1):
7                 if s - f >= 0:
8                     dp[d][s] += dp[d - 1][s - f]
9     return dp[num_dice][target]
10
11 num_dice = int(input("Enter number of dice: "))
12 faces = int(input("Enter faces per dice: "))
13 target = int(input("Enter target distance: "))
14
15 result = dice_ways(num_dice, faces, target)
16 print("Number of ways =", result)
17
```

Output

```
Enter number of dice: 4
Enter faces per dice: 4
Enter target distance: 10
Number of ways = 44

=== Code Execution Successful ===
```

Question 4: Casino Reward Calculation

A casino rewards customers who obtain exactly 12 points by rolling 3 six-faced dice. Write a program to calculate the total number of winning combinations.

Input:

Number of Dice = 3

Faces per Dice = 6

Target Sum = 12

Output:

Number of ways = 25

```
main.py Share Run
1 def dice_ways(num_dice, faces, target):
2     dp = [[0] * (target + 1) for _ in range(num_dice + 1)]
3     dp[0][0] = 1
4     for d in range(1, num_dice + 1):
5         for s in range(1, target + 1):
6             for f in range(1, faces + 1):
7                 if s - f >= 0:
8                     dp[d][s] += dp[d - 1][s - f]
9     return dp[num_dice][target]
10
11 num_dice = int(input("Enter number of dice: "))
12 faces = int(input("Enter faces per dice: "))
13 target = int(input("Enter target sum: "))
14
15 result = dice_ways(num_dice, faces, target)
16 print("Number of ways =", result)
17
```

Output

```
Enter number of dice: 3
Enter faces per dice: 6
Enter target sum: 12
Number of ways = 25

=== Code Execution Successful ===
```

Question 5: Quiz Competition Scoring

In a quiz application, bonus points are awarded if the sum of 5 dice rolls equals 15. Each die has 6 faces. Write a program to find the number of valid scoring combinations.

Input:

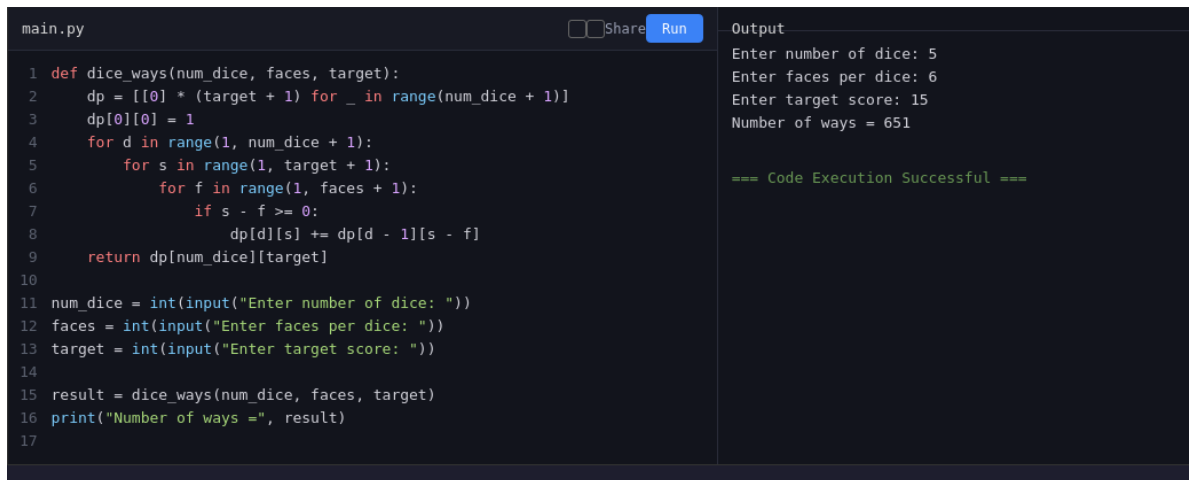
Number of Dice = 5

Faces per Dice = 6

Target Score = 15

Output:

Number of ways = 651



```
main.py ☐ Share 
1 def dice_ways(num_dice, faces, target):
2     dp = [[0] * (target + 1) for _ in range(num_dice + 1)]
3     dp[0][0] = 1
4     for d in range(1, num_dice + 1):
5         for s in range(1, target + 1):
6             for f in range(1, faces + 1):
7                 if s - f >= 0:
8                     dp[d][s] += dp[d - 1][s - f]
9     return dp[num_dice][target]
10
11 num_dice = int(input("Enter number of dice: "))
12 faces = int(input("Enter faces per dice: "))
13 target = int(input("Enter target score: "))
14
15 result = dice_ways(num_dice, faces, target)
16 print("Number of ways =", result)
17
```

Output

```
Enter number of dice: 5
Enter faces per dice: 6
Enter target score: 15
Number of ways = 651

=== Code Execution Successful ===
```

Question 6: Sports Prediction System

Write a program to determine the number of ways a predicted score of 9 can be obtained using 3 five-faced dice.

Input:

Number of Dice = 3

Faces per Dice = 5

Target Score = 9

Output:

Number of ways = 19

```
main.py Share Run
1 def dice_ways(num_dice, faces, target):
2     dp = [[0] * (target + 1) for _ in range(num_dice + 1)]
3     dp[0][0] = 1
4     for d in range(1, num_dice + 1):
5         for s in range(1, target + 1):
6             for f in range(1, faces + 1):
7                 if s - f >= 0:
8                     dp[d][s] += dp[d - 1][s - f]
9     return dp[num_dice][target]
10
11 num_dice = int(input("Enter number of dice: "))
12 faces = int(input("Enter faces per dice: "))
13 target = int(input("Enter target score: "))
14
15 result = dice_ways(num_dice, faces, target)
16 print("Number of ways =", result)
17
```

Output

```
Enter number of dice: 3
Enter faces per dice: 5
Enter target score: 9
Number of ways = 19

=== Code Execution Successful ===
```

7. A lottery simulation system uses 2 ten-faced dice. Find the number of possible combinations that result in a sum of 11.

Input:

Number of Dice = 2

Faces per Dice = 10

Target Sum = 11

Output:

Number of ways = 10

```
main.py Share Run
1 def dice_ways(num_dice, faces, target):
2     dp = [[0] * (target + 1) for _ in range(num_dice + 1)]
3     dp[0][0] = 1
4     for d in range(1, num_dice + 1):
5         for s in range(1, target + 1):
6             for f in range(1, faces + 1):
7                 if s - f >= 0:
8                     dp[d][s] += dp[d - 1][s - f]
9     return dp[num_dice][target]
10
11 num_dice = int(input("Enter number of dice: "))
12 faces = int(input("Enter faces per dice: "))
13 target = int(input("Enter target sum: "))
14
15 result = dice_ways(num_dice, faces, target)
16 print("Number of ways =", result)
17
```

Output

```
Enter number of dice: 2
Enter faces per dice: 10
Enter target sum: 11
Number of ways = 10

=== Code Execution Successful ===
```

8. Write a program to find the number of ways to obtain a given sum using N dice, each having M faces.

Input:

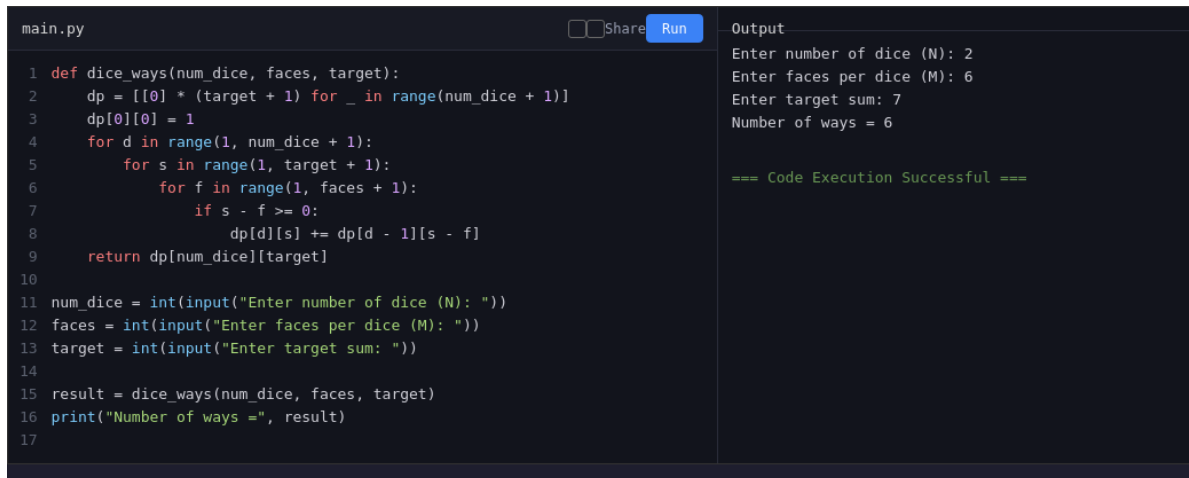
Number of Dice = 2

Faces per Dice = 6

Target Sum = 7

Output:

Number of ways = 6



```
main.py Share Run  
1 def dice_ways(num_dice, faces, target):  
2     dp = [[0] * (target + 1) for _ in range(num_dice + 1)]  
3     dp[0][0] = 1  
4     for d in range(1, num_dice + 1):  
5         for s in range(1, target + 1):  
6             for f in range(1, faces + 1):  
7                 if s - f >= 0:  
8                     dp[d][s] += dp[d - 1][s - f]  
9     return dp[num_dice][target]  
10  
11 num_dice = int(input("Enter number of dice (N): "))  
12 faces = int(input("Enter faces per dice (M): "))  
13 target = int(input("Enter target sum: "))  
14  
15 result = dice_ways(num_dice, faces, target)  
16 print("Number of ways =", result)  
17  
  
Output  
Enter number of dice (N): 2  
Enter faces per dice (M): 6  
Enter target sum: 7  
Number of ways = 6  
  
=== Code Execution Successful ===
```

9. Write a program to determine the number of ways to obtain a target score using multiple dice.

Input:

Number of Dice = 3

Faces per Dice = 6

Target Sum = 8

Output:

Number of ways = 21

```
main.py Share Run
1 def dice_ways(num_dice, faces, target):
2     dp = [[0] * (target + 1) for _ in range(num_dice + 1)]
3     dp[0][0] = 1
4     for d in range(1, num_dice + 1):
5         for s in range(1, target + 1):
6             for f in range(1, faces + 1):
7                 if s - f >= 0:
8                     dp[d][s] += dp[d - 1][s - f]
9     return dp[num_dice][target]
10
11 num_dice = int(input("Enter number of dice: "))
12 faces = int(input("Enter faces per dice: "))
13 target = int(input("Enter target sum: "))
14
15 result = dice_ways(num_dice, faces, target)
16 print("Number of ways =", result)
17
```

Output

```
Enter number of dice: 3
Enter faces per dice: 6
Enter target sum: 8
Number of ways = 21

=== Code Execution Successful ===
```

10. Write a program to count the possible outcomes that produce a specified sum.

Input:

Number of Dice = 4

Faces per Dice = 6

Target Sum = 10

Output:

Number of ways = 80

```
main.py Share Run
1 def dice_ways(num_dice, faces, target):
2     dp = [[0] * (target + 1) for _ in range(num_dice + 1)]
3     dp[0][0] = 1
4     for d in range(1, num_dice + 1):
5         for s in range(1, target + 1):
6             for f in range(1, faces + 1):
7                 if s - f >= 0:
8                     dp[d][s] += dp[d - 1][s - f]
9     return dp[num_dice][target]
10
11 num_dice = int(input("Enter number of dice: "))
12 faces = int(input("Enter faces per dice: "))
13 target = int(input("Enter target sum: "))
14
15 result = dice_ways(num_dice, faces, target)
16 print("Number of ways =", result)
17
```

Output

```
Enter number of dice: 4
Enter faces per dice: 6
Enter target sum: 10
Number of ways = 80

=== Code Execution Successful ===
```

11. Write a program to find the number of possible dice combinations that yield the required score.

Input:

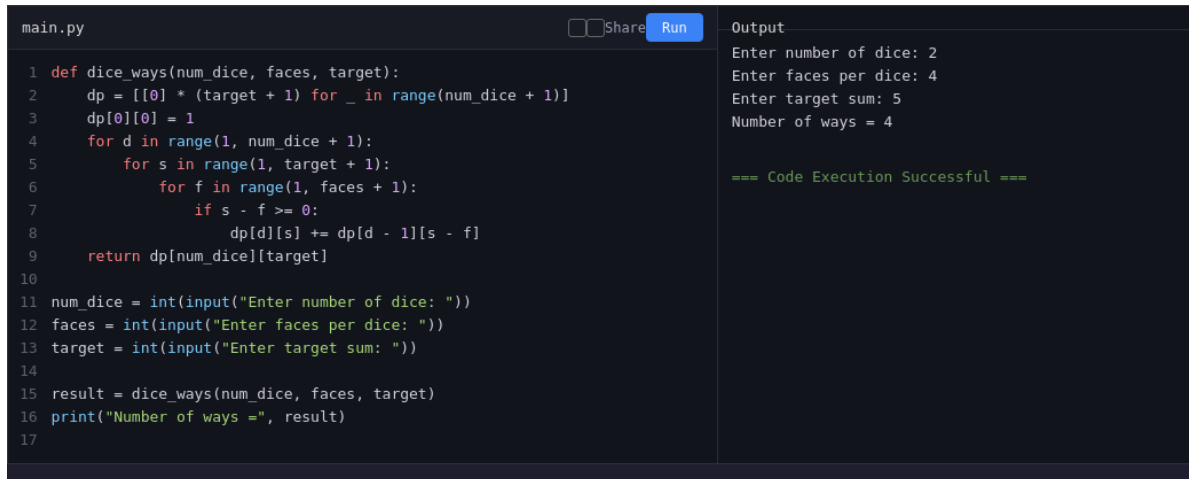
Number of Dice = 2

Faces per Dice = 4

Target Sum = 5

Output:

Number of ways = 4



The screenshot shows a Python IDE with a file named 'main.py'. The code defines a function 'dice_ways' that uses dynamic programming to calculate the number of ways to obtain a target sum using a given number of dice with a specific number of faces. The function initializes a 2D array 'dp' and iterates over the number of dice and the target sum. The main part of the program takes user input for the number of dice, faces per dice, and the target sum, then calls the 'dice_ways' function and prints the result.

```
1 def dice_ways(num_dice, faces, target):
2     dp = [[0] * (target + 1) for _ in range(num_dice + 1)]
3     dp[0][0] = 1
4     for d in range(1, num_dice + 1):
5         for s in range(1, target + 1):
6             for f in range(1, faces + 1):
7                 if s - f >= 0:
8                     dp[d][s] += dp[d - 1][s - f]
9     return dp[num_dice][target]
10
11 num_dice = int(input("Enter number of dice: "))
12 faces = int(input("Enter faces per dice: "))
13 target = int(input("Enter target sum: "))
14
15 result = dice_ways(num_dice, faces, target)
16 print("Number of ways =", result)
17
```

The output of the program is displayed on the right side of the IDE:

```
Output
Enter number of dice: 2
Enter faces per dice: 4
Enter target sum: 5
Number of ways = 4

=== Code Execution Successful ===
```

Binomial Coefficient

1. Write a program to determine the number of ways to obtain the target sum using dice throws.

Input:

Number of Dice = 3

Faces per Dice = 4

Target Sum = 7

Output:

Number of ways = 12


```
main.py Share Run
1 def dice_ways(num_dice, faces, target):
2     dp = [[0] * (target + 1) for _ in range(num_dice + 1)]
3     dp[0][0] = 1
4     for d in range(1, num_dice + 1):
5         for s in range(1, target + 1):
6             for f in range(1, faces + 1):
7                 if s - f >= 0:
8                     dp[d][s] += dp[d - 1][s - f]
9     return dp[num_dice][target]
10
11 num_dice = int(input("Enter number of dice: "))
12 faces = int(input("Enter faces per dice: "))
13 target = int(input("Enter target sum: "))
14
15 result = dice_ways(num_dice, faces, target)
16 print("Number of ways =", result)
17
```

Output

```
Enter number of dice: 3
Enter faces per dice: 4
Enter target sum: 7
Number of ways = 12

=== Code Execution Successful ===
```

2. Write a program to calculate the Binomial Coefficient $C(n, r)$.

Input:

$n = 5$

$r = 2$

Output:

$C(5, 2) = 10$

```
main.py Share Run
1 def binomial_coefficient(n, r):
2     C = [[0] * (r + 1) for _ in range(n + 1)]
3     for i in range(n + 1):
4         for j in range(min(i, r) + 1):
5             if j == 0 or j == i:
6                 C[i][j] = 1
7             else:
8                 C[i][j] = C[i - 1][j - 1] + C[i - 1][j]
9     return C[n][r]
10
11 n = int(input("Enter n: "))
12 r = int(input("Enter r: "))
13
14 result = binomial_coefficient(n, r)
15 print("C(5,2) =", result)
16
```

Output

```
Enter n: 5
Enter r: 2
C(5,2) = 10

=== Code Execution Successful ===
```

3. Write a program to find the number of ways to select r items from n items.

Input:

$n = 8$

$r = 3$

Output:

$$C(8,3) = 56$$

```
main.py ☐ Share 
1 def binomial_coefficient(n, r):
2     C = [[0] * (r + 1) for _ in range(n + 1)]
3     for i in range(n + 1):
4         for j in range(min(i, r) + 1):
5             if j == 0 or j == i:
6                 C[i][j] = 1
7             else:
8                 C[i][j] = C[i - 1][j - 1] + C[i - 1][j]
9     return C[n][r]
10
11 n = int(input("Enter n: "))
12 r = int(input("Enter r: "))
13
14 result = binomial_coefficient(n, r)
15 print("C(8,3) =", result)
16
```

Output

```
Enter n: 8
Enter r: 3
C(8,3) = 56

=== Code Execution Successful ===
```

4

Write a program to compute the Binomial Coefficient using Dynamic Programming.

Input:

$$n = 10$$

$$r = 4$$

Output:

$$C(10,4) = 210$$

```
main.py ☐ Share 
1 def binomial_coefficient(n, r):
2     C = [[0] * (r + 1) for _ in range(n + 1)]
3     for i in range(n + 1):
4         for j in range(min(i, r) + 1):
5             if j == 0 or j == i:
6                 C[i][j] = 1
7             else:
8                 C[i][j] = C[i - 1][j - 1] + C[i - 1][j]
9     return C[n][r]
10
11 n = int(input("Enter n: "))
12 r = int(input("Enter r: "))
13
14 result = binomial_coefficient(n, r)
15 print("C(10,4) =", result)
16
```

Output

```
Enter n: 10
Enter r: 4
C(10,4) = 210

=== Code Execution Successful ===
```

5. Write a program to determine the number of possible committees formed from a group.

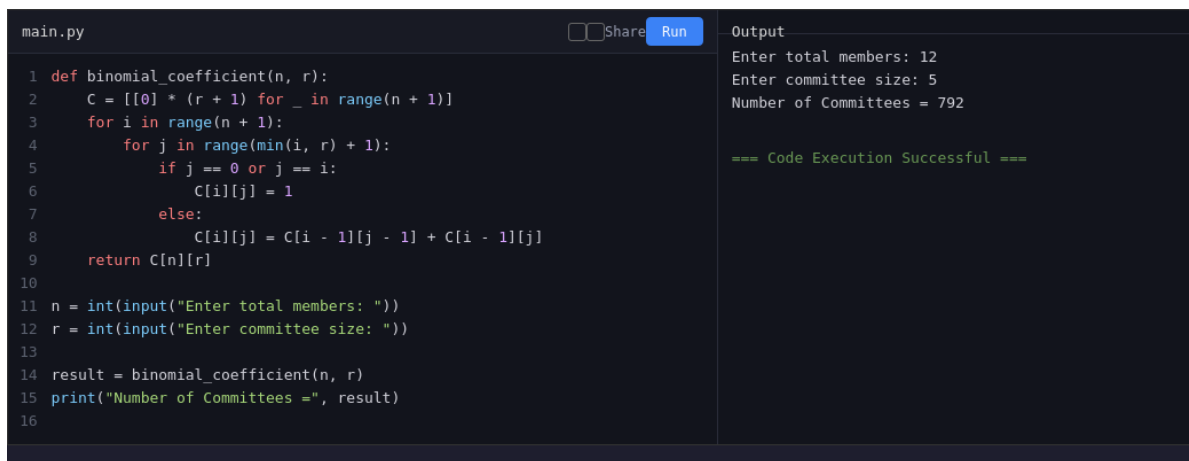
Input:

Total Members = 12

Committee Size = 5

Output:

Number of Committees = 792



```
main.py ☐ Share 
1 def binomial_coefficient(n, r):
2     C = [[0] * (r + 1) for _ in range(n + 1)]
3     for i in range(n + 1):
4         for j in range(min(i, r) + 1):
5             if j == 0 or j == i:
6                 C[i][j] = 1
7             else:
8                 C[i][j] = C[i - 1][j - 1] + C[i - 1][j]
9     return C[n][r]
10
11 n = int(input("Enter total members: "))
12 r = int(input("Enter committee size: "))
13
14 result = binomial_coefficient(n, r)
15 print("Number of Committees =", result)
16
```

Output

```
Enter total members: 12
Enter committee size: 5
Number of Committees = 792

=== Code Execution Successful ===
```

6. Write a program to calculate the combination value for given n and r.

Input:

n = 15

r = 6

Output:

$C(15,6) = 5005$

```
main.py Share Run
1 def binomial_coefficient(n, r):
2     C = [[0] * (r + 1) for _ in range(n + 1)]
3     for i in range(n + 1):
4         for j in range(min(i, r) + 1):
5             if j == 0 or j == i:
6                 C[i][j] = 1
7             else:
8                 C[i][j] = C[i - 1][j - 1] + C[i - 1][j]
9     return C[n][r]
10
11 n = int(input("Enter n: "))
12 r = int(input("Enter r: "))
13
14 result = binomial_coefficient(n, r)
15 print("C(15,6) =", result)
16
```

Output

```
Enter n: 15
Enter r: 6
C(15,6) = 5005

=== Code Execution Successful ===
```

7. A software company has 10 developers and wants to select 3 developers for a special AI project. Write a program to calculate the number of different teams that can be formed.

Input:

Total Developers = 10

Developers Required = 3

Output:

Number of Teams = 120

```
main.py Share Run
1 def binomial_coefficient(n, r):
2     C = [[0] * (r + 1) for _ in range(n + 1)]
3     for i in range(n + 1):
4         for j in range(min(i, r) + 1):
5             if j == 0 or j == i:
6                 C[i][j] = 1
7             else:
8                 C[i][j] = C[i - 1][j - 1] + C[i - 1][j]
9     return C[n][r]
10
11 n = int(input("Enter total developers: "))
12 r = int(input("Enter developers required: "))
13
14 result = binomial_coefficient(n, r)
15 print("Number of Teams =", result)
16
```

Output

```
Enter total developers: 10
Enter developers required: 3
Number of Teams = 120

=== Code Execution Successful ===
```

8. A university must select 4 faculty members from a pool of 12 professors to form a scholarship committee. Write a program to determine the total number of possible committees.

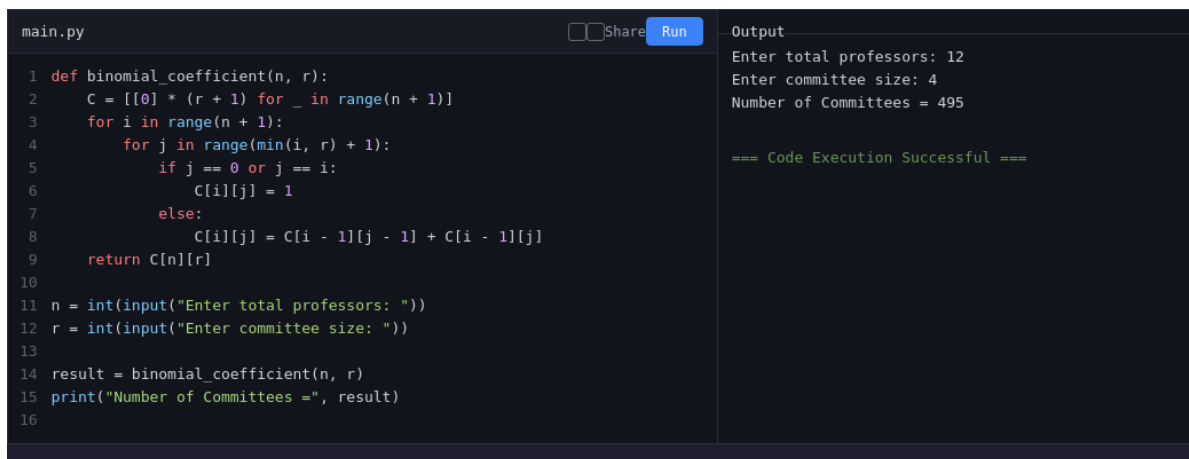
Input:

Total Professors = 12

Committee Size = 4

Output:

Number of Committees = 495



```
main.py Share Run  
1 def binomial_coefficient(n, r):  
2     c = [[0] * (r + 1) for _ in range(n + 1)]  
3     for i in range(n + 1):  
4         for j in range(min(i, r) + 1):  
5             if j == 0 or j == i:  
6                 c[i][j] = 1  
7             else:  
8                 c[i][j] = c[i - 1][j - 1] + c[i - 1][j]  
9     return c[n][r]  
10  
11 n = int(input("Enter total professors: "))  
12 r = int(input("Enter committee size: "))  
13  
14 result = binomial_coefficient(n, r)  
15 print("Number of Committees =", result)  
16  
Output  
Enter total professors: 12  
Enter committee size: 4  
Number of Committees = 495  
  
=== Code Execution Successful ===
```

9. A cricket academy has 15 players and needs to choose 5 players for a tournament squad. Calculate the number of possible selections.

Input:

Total Players = 15

Players Required = 5

Output:

Number of Selections = 3003

```
main.py Share Run
1 def binomial_coefficient(n, r):
2     C = [[0] * (r + 1) for _ in range(n + 1)]
3     for i in range(n + 1):
4         for j in range(min(i, r) + 1):
5             if j == 0 or j == i:
6                 C[i][j] = 1
7             else:
8                 C[i][j] = C[i - 1][j - 1] + C[i - 1][j]
9     return C[n][r]
10
11 n = int(input("Enter total players: "))
12 r = int(input("Enter players required: "))
13
14 result = binomial_coefficient(n, r)
15 print("Number of Selections =", result)
16
```

Output

```
Enter total players: 15
Enter players required: 5
Number of Selections = 3003

=== Code Execution Successful ===
```

10. A company wants to select 6 customers from 20 volunteers to test a new product. Write a program to determine the total number of testing groups.

Input:

Total Volunteers = 20

Group Size = 6

Output:

Number of Groups = 38760

```
main.py Share Run
1 def binomial_coefficient(n, r):
2     C = [[0] * (r + 1) for _ in range(n + 1)]
3     for i in range(n + 1):
4         for j in range(min(i, r) + 1):
5             if j == 0 or j == i:
6                 C[i][j] = 1
7             else:
8                 C[i][j] = C[i - 1][j - 1] + C[i - 1][j]
9     return C[n][r]
10
11 n = int(input("Enter total volunteers: "))
12 r = int(input("Enter group size: "))
13
14 result = binomial_coefficient(n, r)
15 print("Number of Groups =", result)
16
```

Output

```
Enter total volunteers: 20
Enter group size: 6
Number of Groups = 38760

=== Code Execution Successful ===
```

11. A cloud provider has 8 servers and must allocate 3 servers to a customer. Determine the number of possible server combinations.

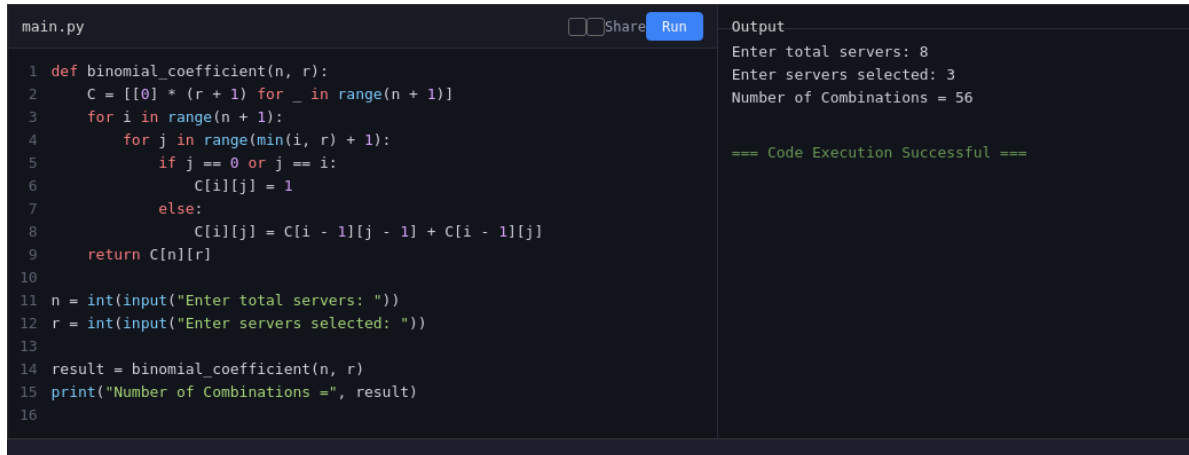
Input:

Total Servers = 8

Servers Selected = 3

Output:

Number of Combinations = 56



```
main.py ☐ Share 
1 def binomial_coefficient(n, r):
2     C = [[0] * (r + 1) for _ in range(n + 1)]
3     for i in range(n + 1):
4         for j in range(min(i, r) + 1):
5             if j == 0 or j == i:
6                 C[i][j] = 1
7             else:
8                 C[i][j] = C[i - 1][j - 1] + C[i - 1][j]
9     return C[n][r]
10
11 n = int(input("Enter total servers: "))
12 r = int(input("Enter servers selected: "))
13
14 result = binomial_coefficient(n, r)
15 print("Number of Combinations =", result)
16
```

Output

```
Enter total servers: 8
Enter servers selected: 3
Number of Combinations = 56

=== Code Execution Successful ===
```

Assembly Line Scheduling

1. Write a program to find the minimum time required to manufacture a product using two assembly lines.

Input:

Entry Times = [10,12]

Exit Times = [18,7]

Line1 = [4,5,3,2]

Line2 = [2,10,1,4]

Transfer1 = [7,4,5]

Transfer2 = [9,2,8]

Output:

Minimum Production Time = 35

```
main.py Share Run Output
1 def min_assembly_time(entry, exit_times, line1, line2, transfer1, transfer2):
2     n = len(line1)
3     T1 = [0] * n
4     T2 = [0] * n
5     T1[0] = entry[0] + line1[0]
6     T2[0] = entry[1] + line2[0]
7     for i in range(1, n):
8         T1[i] = min(T1[i-1] + line1[i], T2[i-1] + transfer2[i-1] + line1[i])
9         T2[i] = min(T2[i-1] + line2[i], T1[i-1] + transfer1[i-1] + line2[i])
10    return min(T1[n-1] + exit_times[0], T2[n-1] + exit_times[1])
11
12 entry_times = [10, 12]
13 exit_times = [18, 7]
14 line1 = [4, 5, 3, 2]
15 line2 = [2, 10, 1, 4]
16 transfer1 = [7, 4, 5]
17 transfer2 = [9, 2, 8]
18
19 result = min_assembly_time(entry_times, exit_times, line1, line2, transfer1, transfer2)
20 print("Minimum Production Time =", result)
21
```

Minimum Production Time = 35

=== Code Execution Successful ===

2. Write a program to determine the optimal assembly line path with minimum processing time.

Input:

Entry Times = [8,10]

Exit Times = [12,15]

Line1 = [5,6,4]

Line2 = [7,3,5]

Transfer1 = [2,1]

Transfer2 = [3,2]

Output:

Minimum Production Time = 33


```
main.py Share Run Output
1 def min_assembly_time(entry, exit_times, line1, line2, transfer1, transfer2):
2     n = len(line1)
3     T1 = [0] * n
4     T2 = [0] * n
5     T1[0] = entry[0] + line1[0]
6     T2[0] = entry[1] + line2[0]
7     for i in range(1, n):
8         T1[i] = min(T1[i-1] + line1[i], T2[i-1] + transfer2[i-1] + line1[i])
9         T2[i] = min(T2[i-1] + line2[i], T1[i-1] + transfer1[i-1] + line2[i])
10    return min(T1[n-1] + exit_times[0], T2[n-1] + exit_times[1])
11
12 entry_times = [8, 10]
13 exit_times = [12, 15]
14 line1 = [5, 6, 4]
15 line2 = [7, 3, 5]
16 transfer1 = [2, 1]
17 transfer2 = [3, 2]
18
19 result = min_assembly_time(entry_times, exit_times, line1, line2, transfer1, transfer2)
20 print("Minimum Production Time =", result)
21
```

Minimum Production Time = 35

=== Code Execution Successful ===

3. Write a program to compute the fastest route through assembly stations.

Input:

Entry Times = [5,6]

Exit Times = [4,5]

Line1 = [3,4,5]

Line2 = [4,3,6]

Transfer1 = [2,1]

Transfer2 = [1,2]

Output:

Minimum Production Time = 21

```
main.py Share Run Output
1 def min_assembly_time(entry, exit_times, line1, line2, transfer1, transfer2):
2     n = len(line1)
3     T1 = [0] * n
4     T2 = [0] * n
5     T1[0] = entry[0] + line1[0]
6     T2[0] = entry[1] + line2[0]
7     for i in range(1, n):
8         T1[i] = min(T1[i-1] + line1[i], T2[i-1] + transfer2[i-1] + line1[i])
9         T2[i] = min(T2[i-1] + line2[i], T1[i-1] + transfer1[i-1] + line2[i])
10    return min(T1[n-1] + exit_times[0], T2[n-1] + exit_times[1])
11
12 entry_times = [5, 6]
13 exit_times = [4, 5]
14 line1 = [3, 4, 5]
15 line2 = [4, 3, 6]
16 transfer1 = [2, 1]
17 transfer2 = [1, 2]
18
19 result = min_assembly_time(entry_times, exit_times, line1, line2, transfer1, transfer2)
20 print("Minimum Production Time =", result)
21
```

Minimum Production Time = 21

=== Code Execution Successful ===

4. Write a program to identify the assembly line schedule that minimizes total manufacturing time.

Input:

Entry Times = [7,9]

Exit Times = [8,6]

Line1 = [4,3,5,2]

Line2 = [5,2,4,3]

Transfer1 = [1,2,1]

Transfer2 = [2,1,2]

Output:

Minimum Production Time = 29

```
main.py □□ Share Run Output  
1 def min_assembly_time(entry, exit_times, line1, line2, transfer1, transfer2):  
2     n = len(line1)  
3     T1 = [0] * n  
4     T2 = [0] * n  
5     T1[0] = entry[0] + line1[0]  
6     T2[0] = entry[1] + line2[0]  
7     for i in range(1, n):  
8         T1[i] = min(T1[i-1] + line1[i], T2[i-1] + transfer2[i-1] + line1[i])  
9         T2[i] = min(T2[i-1] + line2[i], T1[i-1] + transfer1[i-1] + line2[i])  
10    return min(T1[n-1] + exit_times[0], T2[n-1] + exit_times[1])  
11  
12 entry_times = [7, 9]  
13 exit_times = [8, 6]  
14 line1 = [4, 3, 5, 2]  
15 line2 = [5, 2, 4, 3]  
16 transfer1 = [1, 2, 1]  
17 transfer2 = [2, 1, 2]  
18  
19 result = min_assembly_time(entry_times, exit_times, line1, line2, transfer1, transfer2)  
20 print("Minimum Production Time =", result)  
21
```

Minimum Production Time = 27

=== Code Execution Successful ===

5. Write a program to determine the minimum cost path through an assembly system.

Input:

Entry Times = [6,7]

Exit Times = [5,4]

Line1 = [2,4,3]

Line2 = [3,2,5]

Transfer1 = [1,2]

Transfer2 = [2,1]

Output:

Minimum Production Time = 18

```
main.py Share Run Output
1 def min_assembly_time(entry, exit_times, line1, line2, transfer1, transfer2):
2     n = len(line1)
3     T1 = [0] * n
4     T2 = [0] * n
5     T1[0] = entry[0] + line1[0]
6     T2[0] = entry[1] + line2[0]
7     for i in range(1, n):
8         T1[i] = min(T1[i-1] + line1[i], T2[i-1] + transfer2[i-1] + line1[i])
9         T2[i] = min(T2[i-1] + line2[i], T1[i-1] + transfer1[i-1] + line2[i])
10    return min(T1[n-1] + exit_times[0], T2[n-1] + exit_times[1])
11
12 entry_times = [6, 7]
13 exit_times = [5, 4]
14 line1 = [2, 4, 3]
15 line2 = [3, 2, 5]
16 transfer1 = [1, 2]
17 transfer2 = [2, 1]
18
19 result = min_assembly_time(entry_times, exit_times, line1, line2, transfer1, transfer2)
20 print("Minimum Production Time =", result)
21
```

Minimum Production Time = 20

=== Code Execution Successful ===

6. An automobile manufacturer has two assembly lines for producing cars. Each line contains four stations. The processing times at each station and transfer times between lines are provided. Write a program to determine the minimum time required to manufacture a car.

Input:

Entry Times = [10,12]

Exit Times = [18,7]

Line1 = [4,5,3,2]

Line2 = [2,10,1,4]

Transfer1 = [7,4,5]

Transfer2 = [9,2,8]

Output:

Minimum Production Time = 35

```
main.py Share Run Output
1 def min_assembly_time(entry, exit_times, line1, line2, transfer1, transfer2):
2     n = len(line1)
3     T1 = [0] * n
4     T2 = [0] * n
5     T1[0] = entry[0] + line1[0]
6     T2[0] = entry[1] + line2[0]
7     for i in range(1, n):
8         T1[i] = min(T1[i-1] + line1[i], T2[i-1] + transfer2[i-1] + line1[i])
9         T2[i] = min(T2[i-1] + line2[i], T1[i-1] + transfer1[i-1] + line2[i])
10    return min(T1[n-1] + exit_times[0], T2[n-1] + exit_times[1])
11
12 entry_times = [10, 12]
13 exit_times = [18, 7]
14 line1 = [4, 5, 3, 2]
15 line2 = [2, 10, 1, 4]
16 transfer1 = [7, 4, 5]
17 transfer2 = [9, 2, 8]
18
19 result = min_assembly_time(entry_times, exit_times, line1, line2, transfer1, transfer2)
20 print("Minimum Production Time =", result)
21
```

Minimum Production Time = 35

=== Code Execution Successful ===

7. A smartphone company assembles devices using two production lines. Components may switch lines after completing any station. Determine the optimal route that minimizes total production time.

Input:

Entry Times = [8,10]

Exit Times = [12,15]

Line1 = [5,6,4]

Line2 = [7,3,5]

Transfer1 = [2,1]

Transfer2 = [3,2]

Output:

Minimum Production Time = 33

```
main.py Share Run Output
1 def min_assembly_time(entry, exit_times, line1, line2, transfer1, transfer2):
2     n = len(line1)
3     T1 = [0] * n
4     T2 = [0] * n
5     T1[0] = entry[0] + line1[0]
6     T2[0] = entry[1] + line2[0]
7     for i in range(1, n):
8         T1[i] = min(T1[i-1] + line1[i], T2[i-1] + transfer2[i-1] + line1[i])
9         T2[i] = min(T2[i-1] + line2[i], T1[i-1] + transfer1[i-1] + line2[i])
10    return min(T1[n-1] + exit_times[0], T2[n-1] + exit_times[1])
11
12 entry_times = [8, 10]
13 exit_times = [12, 15]
14 line1 = [5, 6, 4]
15 line2 = [7, 3, 5]
16 transfer1 = [2, 1]
17 transfer2 = [3, 2]
18
19 result = min_assembly_time(entry_times, exit_times, line1, line2, transfer1, transfer2)
20 print("Minimum Production Time =", result)
21
```

Minimum Production Time = 35

=== Code Execution Successful ===

8. A food processing company uses two packaging lines. Products pass through labeling, sealing, and packing stations. Write a program to find the minimum completion time.

Input:

Entry Times = [5,6]

Exit Times = [4,5]

Line1 = [3,4,5]

Line2 = [4,3,6]

Transfer1 = [2,1]

Transfer2 = [1,2]

Output:

Minimum Production Time = 21

```
main.py Share Run Output
1 def min_assembly_time(entry, exit_times, line1, line2, transfer1, transfer2):
2     n = len(line1)
3     T1 = [0] * n
4     T2 = [0] * n
5     T1[0] = entry[0] + line1[0]
6     T2[0] = entry[1] + line2[0]
7     for i in range(1, n):
8         T1[i] = min(T1[i-1] + line1[i], T2[i-1] + transfer2[i-1] + line1[i])
9         T2[i] = min(T2[i-1] + line2[i], T1[i-1] + transfer1[i-1] + line2[i])
10    return min(T1[n-1] + exit_times[0], T2[n-1] + exit_times[1])
11
12 entry_times = [5, 6]
13 exit_times = [4, 5]
14 line1 = [3, 4, 5]
15 line2 = [4, 3, 6]
16 transfer1 = [2, 1]
17 transfer2 = [1, 2]
18
19 result = min_assembly_time(entry_times, exit_times, line1, line2, transfer1, transfer2)
20 print("Minimum Production Time =", result)
21
```

Minimum Production Time = 21

=== Code Execution Successful ===

9. Medicine bottles pass through filling, capping, labeling, and packaging stations on two assembly lines. Determine the optimal production schedule.

Input:

Entry Times = [7,9]

Exit Times = [8,6]

Line1 = [4,3,5,2]

Line2 = [5,2,4,3]

Transfer1 = [1,2,1]

Transfer2 = [2,1,2]

Output:

Minimum Production Time = 29

```
main.py Share Run Output
1 def min_assembly_time(entry, exit_times, line1, line2, transfer1, transfer2):
2     n = len(line1)
3     T1 = [0] * n
4     T2 = [0] * n
5     T1[0] = entry[0] + line1[0]
6     T2[0] = entry[1] + line2[0]
7     for i in range(1, n):
8         T1[i] = min(T1[i-1] + line1[i], T2[i-1] + transfer2[i-1] + line1[i])
9         T2[i] = min(T2[i-1] + line2[i], T1[i-1] + transfer1[i-1] + line2[i])
10    return min(T1[n-1] + exit_times[0], T2[n-1] + exit_times[1])
11
12 entry_times = [7, 9]
13 exit_times = [8, 6]
14 line1 = [4, 3, 5, 2]
15 line2 = [5, 2, 4, 3]
16 transfer1 = [1, 2, 1]
17 transfer2 = [2, 1, 2]
18
19 result = min_assembly_time(entry_times, exit_times, line1, line2, transfer1, transfer2)
20 print("Minimum Production Time =", result)
21
```

Minimum Production Time = 27

=== Code Execution Successful ===

10.A circuit board manufacturing facility uses two parallel assembly lines. Products can switch lines between stations. Find the minimum manufacturing time.

Input:

Entry Times = [6,7]

Exit Times = [5,4]

Line1 = [2,4,3]

Line2 = [3,2,5]

Transfer1 = [1,2]

Transfer2 = [2,1]

Output:

Minimum Production Time = 18

main.py

Share Run

Output

```
1 def min_assembly_time(entry, exit_times, line1, line2, transfer1, transfer2):
2     n = len(line1)
3     T1 = [0] * n
4     T2 = [0] * n
5     T1[0] = entry[0] + line1[0]
6     T2[0] = entry[1] + line2[0]
7     for i in range(1, n):
8         T1[i] = min(T1[i-1] + line1[i], T2[i-1] + transfer2[i-1] + line1[i])
9         T2[i] = min(T2[i-1] + line2[i], T1[i-1] + transfer1[i-1] + line2[i])
10    return min(T1[n-1] + exit_times[0], T2[n-1] + exit_times[1])
11
12 entry_times = [6, 7]
13 exit_times = [5, 4]
14 line1 = [2, 4, 3]
15 line2 = [3, 2, 5]
16 transfer1 = [1, 2]
17 transfer2 = [2, 1]
18
19 result = min_assembly_time(entry_times, exit_times, line1, line2, transfer1, transfer2)
20 print("Minimum Production Time =", result)
21
```

Minimum Production Time = 20

=== Code Execution Successful ===